

UNIDAD 3

Bases de datos distribuidas, seguridad y mantenimiento.

TEMA 1:

Fundamentos de bases de datos distribuidas.

Descripción:

La unidad se orienta al estudio de las Bases de Datos Distribuidas, los estudiantes conocerán los fundamentos de la fragmentación, replicación y transparencia, además las principales arquitecturas de estos sistemas de bases de datos, es importante para la administración conocer los aspectos de seguridad y mantenimiento, destacando la gestión de roles, privilegios, auditorías y planes de respaldo, el objetivo es desarrollar competencias para diseñar y administrar entornos distribuidos, garantizando integridad, disponibilidad y confiabilidad de la información, para aplicar en proyectos tecnológicos y organizacionales que requieren soluciones robustas y seguras en el manejo de datos.

Administración de bases de datos.

PERIODO ACADÉMICO 2026-1S

Tabla de contenido

Tema 1: Fundamentos de bases de datos distribuidas.....	2
1.1.Concepto y Arquitecturas de bases de datos distribuidas.	2
1.2.Fragmentación horizontal y vertical.	7
1.3.Replicación y Transparencia.	12
1.4.Transacciones distribuidas y el protocolo 2PC.	16
Bibliografía.....	23

Concepto

Fragmentación

Replicación

Transacciones

Tema 1: Fundamentos de bases de datos distribuidas.

1.1. Concepto y Arquitecturas de bases de datos distribuidas.

Una base de datos distribuida es un conjunto de datos lógicamente relacionados que se encuentran almacenados en múltiples ubicaciones físicas y que son gestionados de manera coordinada por un sistema de gestión distribuido, su principal característica es que para el usuario se presenta como una única base integrada, aunque en la práctica los datos estén fragmentados o replicados en distintos nodos de una red (Özsu & Valduriez, 2021).

Este modelo busca mejorar la disponibilidad, la escalabilidad y la autonomía de los sistemas, al tiempo que mantiene la coherencia e integridad de la información a través de mecanismos de control distribuidos (Emara et al., 2023).

Arquitecturas de bases de datos distribuidas.

La arquitectura de bases de datos distribuidas define la manera en que se organiza, comunica y gestiona la información entre los distintos nodos que conforman un sistema de datos distribuido (Özsu & Valduriez, 2021). El diseño busca brindar al usuario la percepción de que usa una única base de datos integrada, aunque los datos se encuentren físicamente dispersos, las arquitecturas más estudiadas se encuentran los modelos cliente servidor, sistemas federados y esquemas totalmente distribuidos, cada uno con ventajas y limitaciones en el rendimiento, autonomía y coordinación, entender las arquitecturas es esencial para evaluar la eficiencia y escalabilidad de los entornos distribuidos en escenarios reales (Silberschatz, 2020).

Concepto

Fragmentación

Replicación

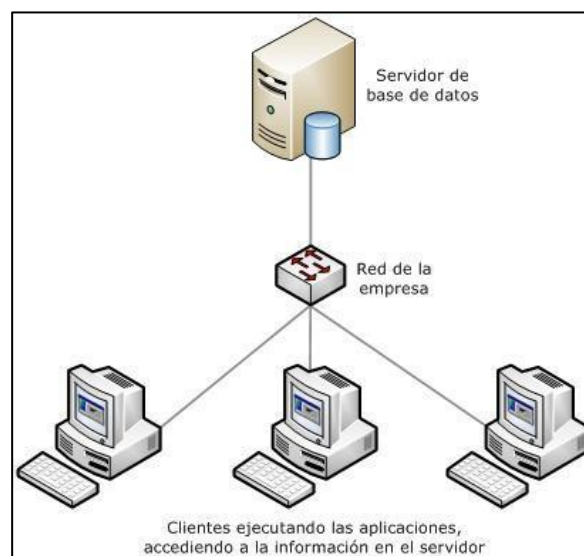
Transacciones

Arquitectura Cliente-Servidor.

En la arquitectura cliente-servidor, la base de datos se gestiona desde un servidor central que atiende las solicitudes de múltiples clientes conectados a través de la red, el servidor es responsable de almacenar, procesar y garantizar la coherencia de los datos, mientras que los clientes se limitan a enviar consultas y recibir resultados, esta arquitectura, aunque concentra el control en un único nodo, facilita la administración y la seguridad, pero puede generar cuellos de botella cuando la demanda es elevada. Es un modelo ampliamente utilizado por su simplicidad y su compatibilidad con la mayoría de los sistemas de gestión de bases de datos (Özsu & Valduriez, 2021).

Figura 1.

Arquitectura Cliente – Servidor.



Arquitectura Federada.

Una base de datos federada integra varios sistemas de bases de datos autónomos que cooperan entre sí, mantiene cada uno su independencia administrativa, en este esquema, cada nodo puede tener su propio gestor y modelo de datos, pero se interconectan

Concepto

Fragmentación

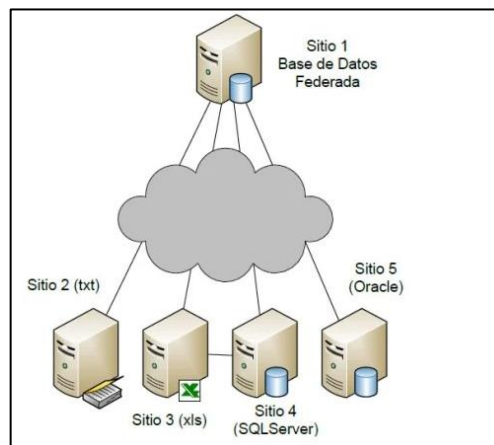
Replicación

Transacciones

mediante un software de federación que permite consultas globales como si fueran parte de una única base, la principal ventaja radica en que no es necesario rediseñar las bases ya existentes, lo que facilita la interoperabilidad entre organizaciones. Sin embargo, la heterogeneidad y las diferencias de control pueden dificultar la consistencia global (Özsu & Valduriez, 2021).

Figura 2.

Arquitectura Federada.

**Arquitectura Peer-to-Peer (P2P).**

En la arquitectura peer-to-peer, no existen servidores centrales, sino que todos los nodos actúan como pares con funciones equivalentes de almacenamiento, procesamiento y comunicación, este modelo favorece la escalabilidad y la tolerancia a fallos, ya que la pérdida de un nodo no interrumpe el funcionamiento general del sistema. Además, permite distribuir la carga de manera más equilibrada entre los participantes. Coordinar transacciones y mantener la coherencia de los datos en un entorno totalmente descentralizado supone un reto técnico mayor, este tipo de arquitectura se utiliza en sistemas modernos que requieren alta disponibilidad y distribución masiva de datos (Özsu & Valduriez, 2021).

Concepto

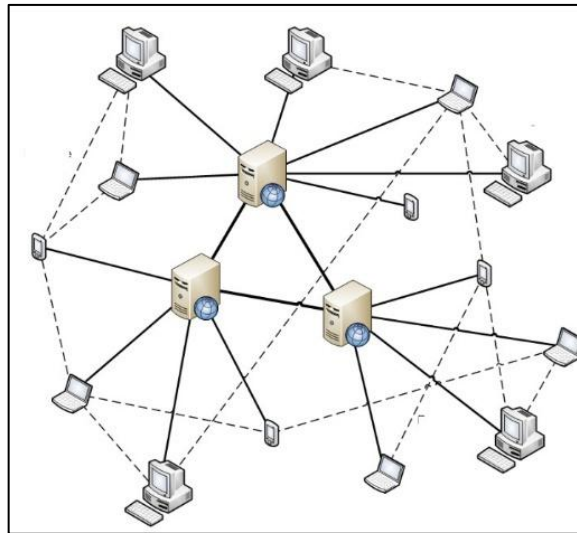
Fragmentación

Replicación

Transacciones

Figura 3.

Arquitectura Peer-to-Peer (P2P).

**Tendencias actuales en arquitecturas distribuidas.**

Las arquitecturas de bases de datos distribuidas han evolucionado hacia esquemas híbridos y soluciones en la nube.

El modelo cliente servidor se encuentra vigente en servicios gestionados como AWS RDS, Azure SQL Database o Google Cloud SQL, estos ofrecen escalabilidad automática y alta disponibilidad, las arquitecturas federadas se utiliza en la integración de fuentes de datos heterogéneas, permite consultas globales en entornos empresariales complejos, por su parte, los sistemas peer to peer han inspirado el desarrollo de bases de datos NoSQL como Cassandra o DynamoDB, ampliamente utilizadas en aplicaciones de Big Data y comercio electrónico. Las tendencias actuales evidencian la necesidad de modelos flexibles capaces de garantizar rendimiento, resiliencia y escalabilidad en contextos de creciente demanda de información (Dong et al., 2024).

Concepto

Fragmentación

Replicación

Transacciones

Tabla 1. Arquitecturas de bases de datos distribuidas

Arquitectura	Características principales	Ventajas	Desventajas
Cliente Servidor	- Servidor central que gestiona la BD. - Clientes envían consultas y reciben resultados. - Control centralizado.	- Administración sencilla. - Seguridad más controlada. - Modelo ampliamente usado y compatible.	- Posible cuello de botella en el servidor. - Escalabilidad limitada. - Dependencia y fuerte del servidor central. - Complejidad para mantener consistencia global.
Federada	- Integración de múltiples BD autónomas. - Cada nodo mantiene su independencia administrativa. - Consultas globales mediante software de federación.	- Permite reutilizar BD existentes. - Favorece interoperabilidad. - No requiere rediseñar sistemas previos.	- Heterogeneidad de modelos y gestores dificulta integración. - Rendimiento variable. - Mayor complejidad en coordinación de transacciones.
Peer to Peer (P2P)	- Todos los nodos son equivalentes. - Distribución de almacenamiento, procesamiento y comunicación. - No existe un servidor central.	- Alta escalabilidad. - Tolerancia a fallos robusta. - Distribución equilibrada de la carga.	- Dificultad para mantener coherencia de datos. - Requiere protocolos avanzados de sincronización.

Concepto

Fragmentación

Replicación

Transacciones

1.2. Fragmentación horizontal y vertical.

La fragmentación en bases de datos distribuidas es la técnica mediante la cual una base de datos se divide en partes más pequeñas llamadas *fragmentos* (Özsu & Valduriez, 2021), con el fin de mejorar la eficiencia, la localización y la autonomía de los datos.

Cada fragmento constituye una porción lógica del conjunto original y puede almacenarse en diferentes nodos de la red, mantiene entre todos la integridad de la base de datos global, el proceso de fragmentación busca optimizar el acceso a la información al acercar los datos a los usuarios o aplicaciones que los requieren con mayor frecuencia, al tiempo que permite distribuir la carga de trabajo y aumentar el rendimiento del sistema (Fuad et al., 2020).

Fragmentación Horizontal.

La **fragmentación horizontal** consiste en dividir una tabla en subconjuntos de filas, de acuerdo con condiciones lógicas que dependen de los valores de los atributos, cada fragmento contiene un subconjunto de tuplas que cumplen un criterio específico, y la unión de todos los fragmentos reconstruye la relación original sin pérdida de información (Özsu & Valduriez, 2021). Esta técnica se utiliza para acercar los datos a los usuarios o aplicaciones que más los requieren, reduce tiempos de respuesta y tráfico en la red (Danach et al., 2024).

Concepto

Fragmentación

Replicación

Transacciones

 Ejemplo 1:

Una empresa multinacional almacena la información de sus empleados en una tabla llamada EMPLEADOS. La tabla contiene los siguientes atributos:

Código SQL

```
-- Tabla Empleados  
EMPLEADOS (ID_Empleado, Nombre, Cargo, País, Salario)
```

Se aplica un entorno distribuido, la empresa decide aplicar fragmentación horizontal para que cada sucursal gestione solo los datos de los empleados que pertenecen a su país.

Se utiliza el atributo País para realizar la fragmentación.

La Sucursal Ecuador **Fragmento 1:**

las tuplas donde País = "Ecuador".

La Sucursal México **Fragmento 2:**

las tuplas donde País = "México".

La Sucursal España **Fragmento 3:**

las tuplas donde País = "España".

Fragmentando de esta manera, cada una de las sucursales accede únicamente a la información de sus empleados, reduce el tráfico de red y mejorando los tiempos de respuesta de las consultas, si fuera necesario reconstruir la tabla completa, bastaría con realizar la unión de todos los fragmentos.

Concepto

Fragmentación

Replicación

Transacciones

Figura 4.

Fragmentación Horizontal en la tabla Empleados.

Tabla original: EMPLEADOS					Fragmento 1: Ecuador				
ID_Empleado	Nombre	Cargo	País	Salario	ID_Empleado	Nombre	Cargo	País	Salario
1	Ana	Analista	Ecuador	1200	1	Ana	Analista	Ecuador	1200
2	Luis	Gerente	México	2500	3	María	Ingeniera	Ecuador	1800
3	María	Ingeniera	Ecuador	1800					
4	José	Técnico	España	1500					
5	Carmen	Administrativa	México	1600					
6	Diego	Consultor	España	2000					
Fragmento 2: México					Fragmento 3: España				
ID_Empleado	Nombre	Cargo	País	Salario	ID_Empleado	Nombre	Cargo	País	Salario
2	Luis	Gerente	México	2500	4	José	Técnico	España	1500
5	Carmen	Administrativa	México	1600	6	Diego	Consultor	España	2000

Fragmentación Vertical.

La fragmentación vertical consiste en dividir una tabla en subconjuntos de columnas o atributos, de manera que cada fragmento conserve siempre el identificador principal de la relación para garantizar la posibilidad de reconstruir la tabla original mediante una operación de unión (*join*) (Özsu & Valduriez, 2021). Este tipo de fragmentación permite distribuir diferentes aspectos de una entidad entre distintos nodos, optimizando el acceso a los datos más utilizados en cada ubicación, de esta forma, cada área de una organización accede únicamente a la información que necesita, reduce el volumen de datos transferidos y mejorando la eficiencia del sistema (Amiri, 2025).

Concepto

Fragmentación

Replicación

Transacciones

 Ejemplo 2:

Se trabaja con la table de empleados y los siguientes atributos:

Código SQL

```
-- Tablas Empleados  
EMPLEADOS (ID_Empleado, Nombre, Dirección, Cargo, Salario)
```

Se aplica fragmentación vertical, divide la tabla en dos fragmentos
Fragmento 1

Código SQL

```
-- atributos  
(ID_Empleado, Nombre, Dirección)
```

El fragmento 1 puede ser administrado por el departamento de recursos humanos, encargado de la información básica de los empleados.

Fragmento 2

Código SQL

```
-- atributos  
(ID_Empleado, Cargo, Salario)
```

El fragmento 2 puede ser administrado por el área contable y financiera, que necesita consultar cargos y remuneraciones.

El atributo **ID_Empleado** se conserva en ambos fragmentos como clave para permitir la reconstrucción de la tabla original mediante una operación de **JOIN**, de esta forma, cada área accede únicamente a la información que necesita, reduce tiempos de consulta y tráfico innecesario en la red.

Concepto

Fragmentación

Replicación

Transacciones

Figura 5.
Fragmentación Vertical en la tabla Empleados.

ID_Empleado	Nombre	Dirección	Cargo	Salario
1	Ana	Quito	Analista	1200
2	Luis	CDMX	Gerente	2500
3	María	Guayaquil	Ingeniera	1800
4	José	Madrid	Técnico	1500

Fragmento 1: Datos Personales

ID_Empleado	Nombre	Dirección
1	Ana	Quito
2	Luis	CDMX
3	María	Guayaquil
4	José	Madrid

Fragmento 2: Datos Laborales

ID_Empleado	Cargo	Salario
1	Analista	1200
2	Gerente	2500
3	Ingeniera	1800
4	Técnico	1500

Tabla 2. Comparación entre Fragmentación Horizontal y Vertical.

Criterio	Fragmentación Horizontal	Fragmentación Vertical
Definición	Divide una tabla en subconjuntos de filas según una condición lógica.	Divide una tabla en subconjuntos de columnas , mantiene siempre el atributo clave.
Unidad de fragmentación	Tuplas o registros completos.	Atributos o columnas.
Objetivo principal	Acercar los datos a los usuarios o aplicaciones que más los necesitan, reduce tráfico de red.	Optimizar el acceso a subconjuntos de atributos específicos según el área de trabajo.

Concepto

Fragmentación

Replicación

Transacciones

Ventajas

- Reduce el tiempo de consulta en cada nodo.
- Mejora el rendimiento en entornos distribuidos.
- Favorece el paralelismo.
- Permite que distintas áreas accedan solo a los datos que necesitan.
- Disminuye el volumen de información transferida.
- Facilita la privacidad de ciertos atributos.

Desventajas

- Puede generar desequilibrio si un nodo concentra más datos que otros.
- Requiere operaciones de unión para reconstruir la tabla.
- Consultas que necesiten muchos atributos requieren uniones frecuentes (*join*).
- Mayor complejidad de mantenimiento.

1.3. Replicación y Transparencia.

Para los sistemas de bases de datos distribuidas, la replicación y la transparencia son mecanismos importantes para garantizar disponibilidad, confiabilidad y facilidad de uso (Özsu & Valduriez, 2021).

Replicación.

La replicación es la técnica que permite mantener varias copias de un mismo fragmento o relación en distintos nodos de una base de datos distribuida, su objetivo es mejorar la disponibilidad de la información, garantizar la tolerancia a fallos y optimizar el acceso de los datos a los usuarios (Özsu & Valduriez, 2021). Existen varios esquemas de replicación como: total, se duplican todos los datos en cada nodo; parcial, solo ciertos fragmentos se replican en ubicaciones específicas; y selectiva, se aplica a datos de alta demanda. La replicación incrementa la confiabilidad del sistema, pero también introduce complejidad, pues

Concepto

Fragmentación

Replicación

Transacciones

requiere mantener la coherencia entre las copias a través de mecanismos de sincronización y control de concurrencia (Zhou et al., 2023).

● Ejemplo 3:

● Replicación Total

Una cadena internacional de supermercados maneja la tabla PRODUCTOS con información sobre nombre, código y precio, para asegurar que cualquier sucursal pueda atender a sus clientes, aunque falle la conexión con la central, se decide replicar la tabla completa en todos los nodos.

- **Ventaja:** en este caso pensemos que la sucursal de Quito pierde conexión con el servidor central, el sistema puede seguir trabajando y consulta los precios porque tiene una copia local.
- **Desventaja:** Para cualquier cambio que se realice en un precio debe sincronizarse en todas las sucursales, esto puede generar más tráfico y riesgo de inconsistencias si la red es lenta.

● Ejemplo 4:

● Replicación Parcial

Un banco regional gestiona la tabla CLIENTES con información de todas sus cuentas, para mejorar el acceso decide replicar solo los clientes VIP en todas las sucursales, mientras que los demás permanecen en su servidor local.

- **Ventaja:** los clientes de alta prioridad son atendidos con rapidez desde cualquier sucursal.
- **Desventaja:** las consultas generales pueden depender del servidor central y tardar más.

Concepto

Fragmentación

Replicación

Transacciones

 Ejemplo 5:**Replicación Selectiva**

Una empresa de envíos internacionales almacena en la tabla PAQUETES los pedidos de todo el mundo, para optimizar los tiempos, los paquetes de América del Sur se replican en nodos ubicados en Brasil y Argentina, mientras que los paquetes de Europa se replican en nodos de España y Alemania.

- **Ventaja:** se mejora el acceso local, reduce la latencia y el tiempo de respuesta.
- **Desventaja:** si un usuario necesita consultar todos los envíos globales, debe acceder a múltiples nodos, lo que complica la coordinación.

Transparencia.

La transparencia en bases de datos distribuidas se refiere a la capacidad del sistema para ocultar al usuario los detalles de la distribución física de los datos, de modo que las operaciones se realicen como si toda la información estuviera en una única base centralizada (Özsu & Valduriez, 2021). Esto significa que el usuario no necesita conocer en qué nodo se encuentran los datos, cómo fueron fragmentados o si existen réplicas. Existen varios tipos de transparencia: de localización, que oculta la ubicación física de los datos; de fragmentación, que permite acceder a la base, aunque esté dividida en fragmentos; de replicación, que asegura que las copias se manejen como una sola; y de concurrencia y fallos, que garantizan operaciones consistentes aún en entornos multiusuario y con interrupciones. Gracias a la transparencia, los sistemas distribuidos ofrecen una experiencia sencilla al usuario, mientras el gestor se encarga de la compleja coordinación interna (Altaher, 2024).

Concepto

Fragmentación

Replicación

Transacciones

Ejemplo 6:

Transparencia de Localización

El usuario realiza una consulta:

Código SQL

```
SELECT * FROM CLIENTES WHERE ID_Cliente = 101;
```

Aunque los datos del cliente 101 se encuentren almacenados físicamente en el nodo de **Guayaquil**, el sistema le devuelve el resultado sin que él sepa en qué servidor se encuentran, el usuario no necesita conocer la ubicación real de los datos.

Ejemplo 7:

Transparencia de Fragmentación

La tabla **EMPLEADOS** está fragmentada horizontalmente por país: Ecuador, México y España. Si un administrador consulta esa tabla:

Código SQL

```
SELECT Nombre, Cargo FROM EMPLEADOS;
```

La base de datos reúne automáticamente los registros de los tres fragmentos y entrega un solo resultado, el usuario que realiza la consulta no percibe que los datos están divididos en fragmentos.

Concepto

Fragmentación

Replicación

Transacciones

 Ejemplo 8:**Transparencia de Replicación**

La tabla PRODUCTOS está replicada en las sucursales de Quito y Cuenca, a un cajero en Cuenca se le pide que consulte el precio de un producto, obtiene el mismo resultado que en Quito, aunque los datos provengan de la réplica local, el sistema asegura que las réplicas sean consistentes y transparentes para el usuario.

 Ejemplo 9:**Transparencia de Concurrencia y Fallos**

Dos usuarios diferentes intentan actualizar simultáneamente el saldo de la misma cuenta bancaria en diferentes nodos, el sistema de gestión distribuye la transacción y garantiza que solo una modificación quede registrada de manera correcta, aunque se presenten múltiples accesos y posibles fallos de red, para el usuario final el sistema se comporta como si fuera una sola base coherente.

1.4. Transacciones distribuidas y el protocolo 2PC.**Transacciones distribuidas.**

Una transacción distribuida es un conjunto de operaciones que se ejecutan de forma coordinada en distintos nodos de una base de datos distribuida, pero que para el usuario se perciben como una sola unidad

Concepto

Fragmentación

Replicación

Transacciones

de trabajo (Özsu & Valduriez, 2021). Estas transacciones mantienen las propiedades ACID: deben ser atómicas (se realizan completamente o no se realizan), consistentes (preservan la integridad de los datos en todos los nodos), aisladas (no interfieren con otras transacciones concurrentes) y duraderas (los cambios persisten incluso ante fallos) (Silberschatz, 2020).

El reto principal es garantizar que las transacciones, a pesar de estar repartidas en diferentes ubicaciones, las operaciones tengan el mismo resultado que si se hubieran ejecutado en un único servidor central, para ello, los sistemas emplean coordinadores y protocolos que aseguran la sincronización y la coherencia entre los nodos (Zhang et al., 2023).

● Ejemplo 10:

Un cliente del Banco del Pacífico realiza una transferencia de 1.000 USD desde su cuenta en la Sucursal Quito hacia otra cuenta en la Sucursal Guayaquil.

- Para esta transacción de trabaja con dos nodos: Quito que debe restar el dinero y Guayaquil que debe sumarlo.
- Si las dos operaciones se completan con éxito, la transacción se confirma en todos los nodos.
- Si uno de los nodos tiene algún fallo, por ejemplo, Quito descuenta, pero Guayaquil no acredita, el sistema ejecuta un rollback para anular la operación en Quito y mantener la consistencia global.

Para el cliente, la operación se percibe como un único proceso, aunque internamente fue distribuido.

Concepto

Fragmentación

Replicación

Transacciones

Tablas

Código SQL

```
--Crear tabla
-- cuentas(id_cuenta PK, saldo >= 0)
CREATE TABLE cuentas (
  id_cuenta INT PRIMARY KEY,
  saldo DECIMAL(12,2) NOT NULL,
  CONSTRAINT saldo_no_negativo CHECK (saldo >= 0)
);
```

Transacción (resta en Quito y suma en Guayaquil):

Código SQL

```
-- 1) Iniciar la transacción
START TRANSACTION;

-- 2) Debitar solo si hay fondos suficientes
UPDATE cuentas
  SET saldo = saldo - 1000
 WHERE id_cuenta = 1001
  AND saldo >= 1000;

-- En el cliente/comando, verifica cuántas filas se afectaron:
-- si fue 0, significa que no había saldo → ROLLBACK;
-- si fue 1, continua.

-- 3) Acreditar en la otra cuenta
UPDATE cuentas
  SET saldo = saldo + 1000
 WHERE id_cuenta = 2002;

-- 4) Confirmar
COMMIT;

-- Si algo falla en cualquier paso (constraint, error, etc.), hacer:
-- ROLLBACK;
```

Atomicidad básica, si algo falla a mitad de la transacción, se realiza un ROLLBACK y deja todo como estaba al iniciar la transacción.

Concepto

Fragmentación

Replicación

Transacciones

Protocolo 2PC (two phase commit).

El protocolo de confirmación en dos fases (2PC) es una técnica clave para lograr transacciones distribuidas en sistemas de almacenamiento como bases de datos relacionales y distribuidas, es el mecanismo más utilizado para garantizar la atomicidad de las transacciones distribuidas, su función es coordinar múltiples nodos participantes de manera que todos confirmen (*commit*) o todos deshagan (*rollback*) en una transacción (Özsu & Valduriez, 2021).

Las dos fases son:

Preparación (votación):

- En esta fase el coordinador envía un mensaje de *PREPARE* a todos los nodos participantes.
- Todos los nodos ejecutan la transacción localmente, pero no la confirma.
- Todos los nodos responden al coordinador con un voto: "YES" (puedo confirmar) o "NO" (no puedo confirmar).

Confirmación (decisión):

- En esta fase si todos los nodos responden "YES", el coordinador envía un mensaje de *COMMIT* y cada nodo confirma la transacción.
- Si algún nodo responde "NO", o sucede un fallo en algún nodo, el coordinador ordena un *ROLLBACK* en todos los nodos.

Al utilizar estas fases, se asegura que la transacción sea atómica globalmente, aunque incrementa el costo en comunicación y pueda verse afectada por bloqueos si el coordinador falla.

Concepto

Fragmentación

Replicación

Transacciones

 Ejemplo 11:

- Un cliente realiza la transferencia de 1000 USD desde su cuenta en la **Sucursal Quito** a otra cuenta en la **Sucursal Guayaquil**.
 - **En la fase 1 de Preparación:**
 - El coordinador envía PREPARE a los nodos Quito y Guayaquil.
 - EL nodo Quito prepara: saldo - 1000 y responde YES.
 - El nodo Guayaquil prepara: saldo + 1000 y responde YES.
 - **En la fase 2 Confirmación:**
 - Como ambos nodos respondieron YES, el coordinador envía COMMIT.
 - Los dos nodos confirman los cambios.
 - Si un nodo respondía NO, ambos nodos realizan un ROLLBACK.

Crear las Tablas

Código SQL

```
-- En QUITO
CREATE TABLE IF NOT EXISTS cuentas(
  id_cuenta INT PRIMARY KEY,
  saldo NUMERIC(12,2) NOT NULL CHECK (saldo >= 0)
);
INSERT INTO cuentas VALUES (1001, 1500.00) ON CONFLICT (id_cuenta) DO
NOTHING;

-- En GUAYAQUIL
CREATE TABLE IF NOT EXISTS cuentas(
  id_cuenta INT PRIMARY KEY,
  saldo NUMERIC(12,2) NOT NULL CHECK (saldo >= 0)
);
INSERT INTO cuentas VALUES (2002, 500.00) ON CONFLICT (id_cuenta) DO
NOTHING;
```

Concepto

Fragmentación

Replicación

Transacciones

Fase 1 PREPARACIÓN (votación)

Fase 1 Nodo Quito

Código SQL

```
--Nodo Quito
-- QUITO: Debitar 1000 si hay saldo
BEGIN;
UPDATE cuentas
  SET saldo = saldo - 1000
 WHERE id_cuenta = 1001
  AND saldo >= 1000;

-- Verifica filas afectadas (en tu cliente/IDE):
-- 1 → OK; 0 → no había saldo → ROLLBACK y vota NO.

-- Si todo OK, PREPARE (queda pendiente de decisión global)
PREPARE TRANSACTION 'tx_quito_1';
```

Fase 1 Nodo GUAYAQUIL

Código SQL

```
--Nodo Guayaquil
-- GUAYAQUIL: Acreditar 1000
BEGIN;
UPDATE cuentas
  SET saldo = saldo + 1000
 WHERE id_cuenta = 2002;

-- Si todo OK:
PREPARE TRANSACTION 'tx_gye_1';
```

Para ver las transacciones preparadas en un nodo:

Código SQL

```
SELECT gid, prepared, owner, database FROM pg_prepared_xacts;
-- Debes ver 'tx_quito_1' en QUITO y 'tx_gye_1' en GUAYAQUIL.
```

Concepto

Fragmentación

Replicación

Transacciones

Fase 2 DECISIÓN (confirmación o abortar)

Caso A: Todo OK, COMMIT global

Código SQL

```
-- Ejecuta en cada nodo:  
COMMIT PREPARED 'tx_quito_1';  
COMMIT PREPARED 'tx_gye_1';
```

Caso B: Falla en algún nodo, ROLLBACK global

Código SQL

```
-- Si QUITO o GUAYAQUIL no pudo preparar o detectaste problema:  
ROLLBACK PREPARED 'tx_quito_1';  
ROLLBACK PREPARED 'tx_gye_1';
```

Verificación

Código SQL

```
-- En QUITO  
SELECT * FROM cuentas WHERE id_cuenta = 1001;  
  
-- En GUAYAQUIL  
SELECT * FROM cuentas WHERE id_cuenta = 2002;
```

Si la transacción realiza un COMMIT global: 1001 tendrá 500.00 y 2002 tendrá 1500.00.

Si la transacción realiza un ROLLBACK global: saldos vuelven a sus valores originales.

Concepto

Fragmentación

Replicación

Transacciones

Bibliografía.

Altaher, R. (2024). *Transparency Levels in Distributed Database Management System DDBMS*. Computer Science and Mathematics. <https://doi.org/10.20944/preprints202404.1327.v1>

Amiri, A. (2025). Maximizing data utility while preserving privacy through database fragmentation. *Expert Systems with Applications*, 273, 126873. <https://doi.org/10.1016/j.eswa.2025.126873>

Arun Kumar Akuthota. (2025). Role-Based Access Control (RBAC) in Modern Cloud Security Governance: An In-depth Analysis. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 11(2), 3297-3311. <https://doi.org/10.32628/CSEIT25112793>

Danach, K., Khalaf, A. H., Rammal, A., & Harb, H. (2024). Enhancing DDBMS Performance through RFO-SVM Optimized Data Fragmentation: A Strategic Approach to Machine Learning Enhanced Systems. *Applied Sciences*, 14(14), 6093. <https://doi.org/10.3390/app14146093>

Dong, H., Zhang, C., Li, G., & Zhang, H. (2024). Cloud-Native Databases: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(12), 7772-7791. <https://doi.org/10.1109/TKDE.2024.3397508>

Elmasri, R., & Navathe, S. (2020). *Fundamentos de sistemas de bases de datos* (5a ed). Pearson Addison Wesley.

Emara, T. Z., Trinh, T., & Huang, J. Z. (2023). Geographically distributed data management to support large-scale data analysis. *Scientific Reports*, 13(1), 17783. <https://doi.org/10.1038/s41598-023-44789-x>

Fuaad, H., Ibrahim, A., Majed, A., & Asem, A. (2020). A Survey on Distributed Databases Fragmentation, Allocation and Replication Algorithms. *Current Journal of Applied Science and Technology*, 27(2), 1-12. <https://doi.org/10.9734/CJAST/2018/37079>

Özsu, M. T. (2022). Database Administrator (DBA). En *Encyclopedia of Database Systems* (pp. 935-936). Springer, New York, NY. https://doi.org/10.1007/978-1-4614-8265-9_80652

Özsu, M. T., & Valduriez, P. (2021). *Principles of Distributed Database Systems, Third Edition*. Springer New York. <https://doi.org/10.1007/978-1-4419-8834-8>

Concepto

Fragmentación

Replicación

Transacciones

Regueiro, C., Seco, I., Gutiérrez-Agüero, I., Urquizu, B., & Mansell, J. (2021). A Blockchain-Based Audit Trail Mechanism: Design and Implementation. *Algorithms*, 14(12), 341. <https://doi.org/10.3390/a14120341>

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (2021). Role-based access control models. *Computer*, 29(2), 38-47. <https://doi.org/10.1109/2.485845>

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (5. ed., international ed). McGraw-Hill Higher Education.

Silberschatz, A. (with Korth, H. F., Sudarshan, S., Sáenz Pérez, F., García Cordero, A., Correas Fernández, J., & Grau Fernández, L.). (2020). *Fundamentos de bases de datos* (5a ed). McGraw-Hill.

Zhang, Q., Li, J., Zhao, H., Xu, Q., Lu, W., Xiao, J., Han, F., Yang, C., & Du, X. (2023). Efficient Distributed Transaction Processing in Heterogeneous Networks. *Proceedings of the VLDB Endowment*, 16(6), 1372-1385. <https://doi.org/10.14778/3583140.3583153>

Zhou, W., Peng, Q., Zhang, Z., Zhang, Y., Ren, Y., Li, S., Fu, G., Cui, Y., Li, Q., Wu, C., Han, S., Wang, S., Li, G., & Yu, G. (2023). GeoGauss: Strongly Consistent and Light-Coordinated OLTP for Geo-Replicated SQL Database. *Proceedings of the ACM on Management of Data*, 1(1), 1-27. <https://doi.org/10.1145/3588916>